



Diyar United Company
PARTNERS IN TECHNOLOGY

Diyar HR Project

Progress Report and Path to Finalization

Prepared for: Diyar Management

Prepared by: Jassem El Danaf

Date: April 2026

Status: Proof of concept, work in progress

An end-to-end internal HR pipeline covering job openings, AI-assisted CV evaluation, shortlisting, and candidate communication. Runs entirely on a local machine with no cloud dependencies.

Contents

1	Executive Summary	2
2	System Architecture	2
3	Infrastructure Layer	2
3.1	Docker and PostgreSQL	2
3.2	n8n as the Orchestration Layer	3
4	Frontend Functionalities	4
4.1	Dashboard	4
4.2	Job Openings	5
4.3	CV Evaluation Wizard	6
4.4	Shortlist and Interview Tracking	8
4.5	Emails	8
5	What Has Been Delivered	9
6	Planned Work Before Finalization	9
6.1	Reliability and Polish	10
6.2	Feature Completion	10
6.3	Quality and Scoring Improvements	10
6.4	Handover Readiness	10
7	Closing Note	10

1. Executive Summary

The Diyar HR project is at a proof-of-concept stage. It is not a finished product yet, but it is a working prototype that an HR user can open in a browser and drive from end to end, covering five connected stages: dashboard, job openings, AI-assisted CV evaluation, shortlisting, and candidate emails. Everything runs locally on one machine, so there are no hosting costs and no candidate data leaves the company. This report summarizes what has been built, how the pieces fit together, and the work still ahead before the project can be considered complete.

2. System Architecture

The solution is built from four cooperating pieces, each with a narrow responsibility. The browser talks only to n8n; n8n talks to the database, the AI model, and a small Python helper that sends emails. This keeps each layer simple and replaceable.

Component	Role
Frontend (web UI)	Single HTML file with all pages, wizards, and tables. The HR user opens it in a browser with nothing to install.
n8n workflows	Visual automation engine that exposes webhooks to the browser, queries the database, and calls the AI model. Plays the role of a backend without needing one written from scratch.
PostgreSQL (Docker)	Stores every job opening, candidate, evaluation, shortlist entry, and email record.
Local AI model (Ollama, qwen3:4b)	Generates job descriptions, drafts evaluation criteria, and scores CVs. All inference runs on the laptop.
SMTP helper (Python)	Small local service that sends outbound email and reports delivery status back to n8n.

Table 1: The five components of the Diyar HR system and what each one is responsible for.

3. Infrastructure Layer

3.1 Docker and PostgreSQL

PostgreSQL runs inside a Docker container named `hr-postgres`. Docker keeps the database isolated from the host machine and makes setup a single command. Anyone who needs to reproduce the environment on a new laptop can do so in minutes.

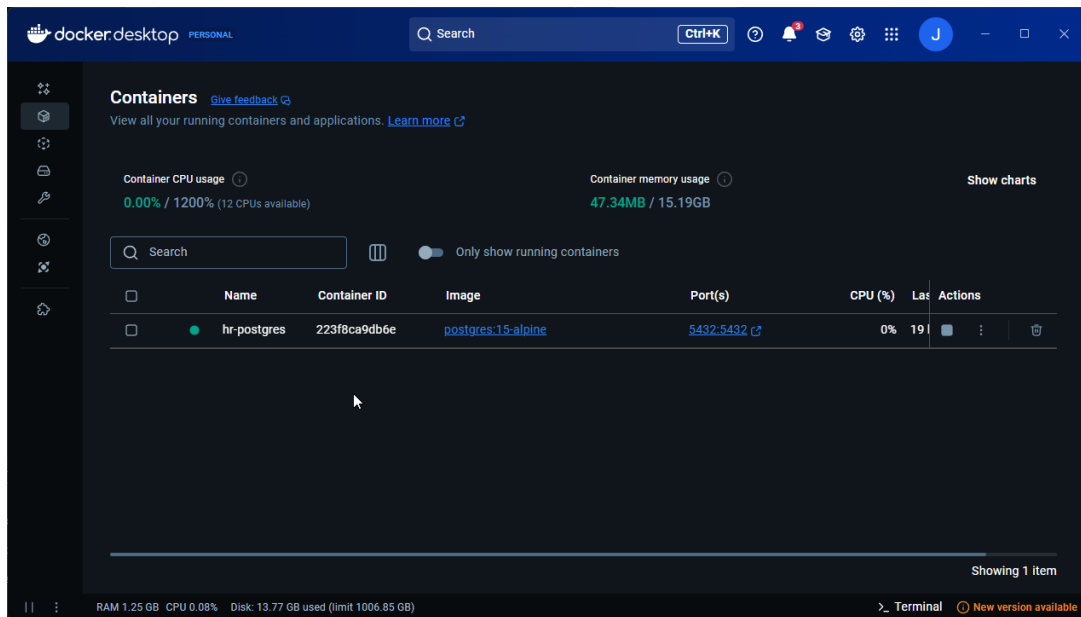


Figure 1: Docker Desktop showing the `hr-postgres` container running on port 5432. This is the only container the project uses, which keeps operations straightforward.

3.2 n8n as the Orchestration Layer

Instead of writing a custom backend from scratch, the project uses n8n. Every browser request hits an n8n webhook, which then runs a small pipeline of steps such as validating input, reading or writing the database, calling the AI model, or sending an email.

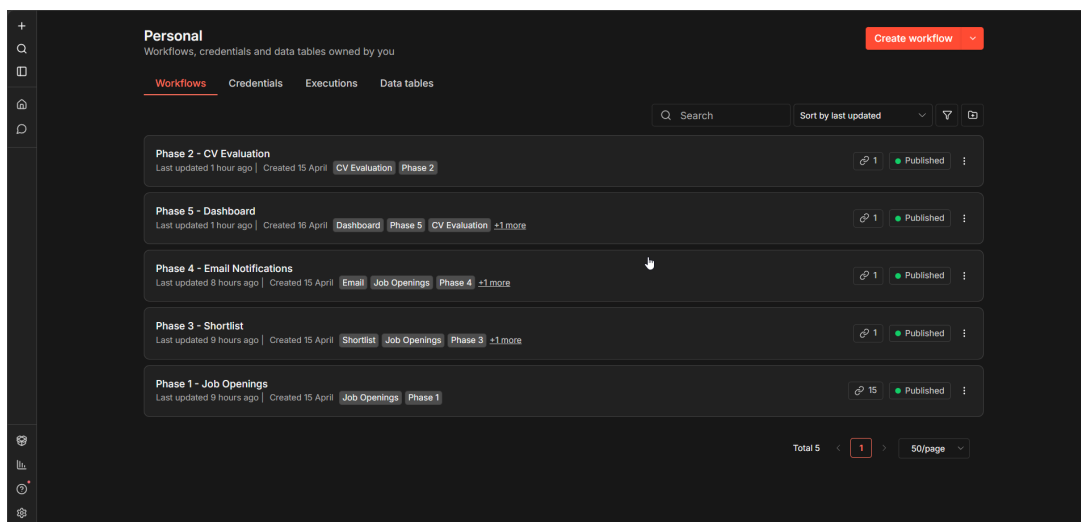


Figure 2: The n8n workspace with the five workflows that power the application. Each workflow is tagged by phase and is currently published and active.



Figure 3: Inside one workflow (Phase 1, Job Openings). Each block is a step such as validation, database access, or response shaping. Editing the logic is visual, which makes future changes faster and easier to review.

4. Frontend Functionalities

The HR user sees one single-page application with five tabs. Selecting a job in any tab carries that selection over to the other tabs through a persistent header badge, so context is never lost.

4.1 Dashboard

The Dashboard gives a snapshot of hiring activity for the selected job. It shows counts of candidates at each stage of the funnel, the average and top evaluation scores, and a status breakdown.

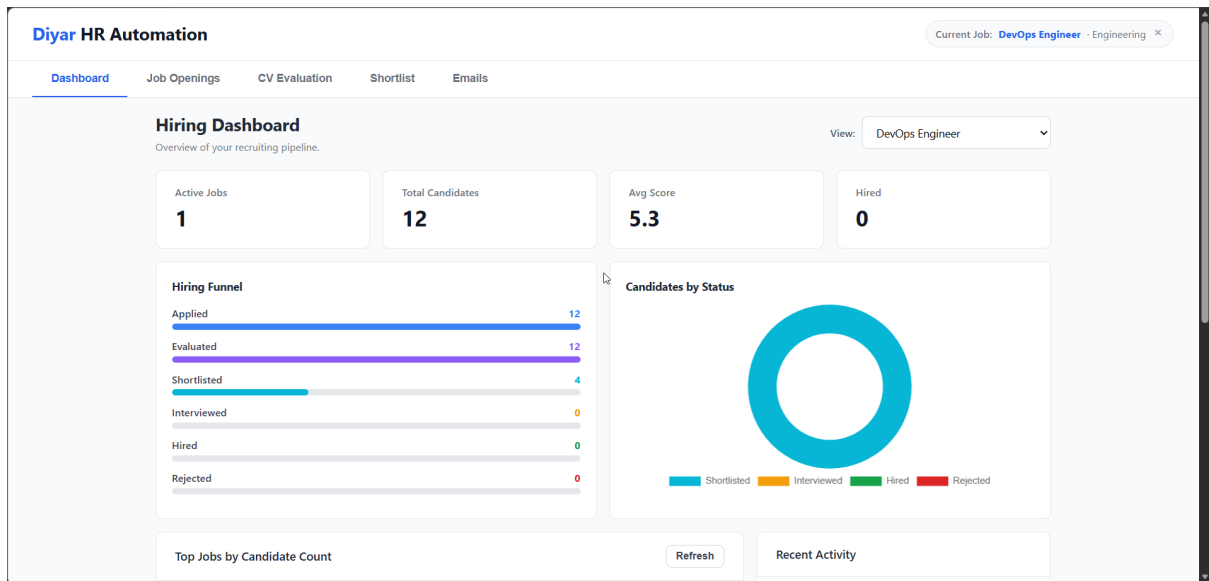


Figure 4: Dashboard view for the DevOps Engineer role. At a glance the user can see 12 candidates applied, 12 were evaluated, 4 were shortlisted, and 8 were rejected.

4.2 Job Openings

Job openings are created through a short two-step wizard. The user enters the basic information first, then provides a job description either by writing it manually, uploading a file, or asking the local AI model to draft one.

Job Openings
2 of 2 jobs

+ New Job

Search by title, department...

All Active Inactive

Job Title	Department / Level	Created	Status	Active	LinkedIn
DevOps Engineer	Engineering - Junior - Full-time	Apr 16, 2026	draft	☒	Publish
Software Engineer	Engineering - Mid-level - Full-time	Apr 15, 2026	draft	☒	Publish

Figure 5: The Job Openings list. Each row can be activated or deactivated with a toggle, and the Publish LinkedIn button marks the job ready for publishing (outbound publishing itself is planned for a later phase).

4.3 CV Evaluation Wizard

The evaluation flow is a four-step wizard: choose the job, set the scoring criteria, upload CVs, and review results. The steps are revisitable at any time, so the user can adjust criteria and re-evaluate without starting over.

Step 2a, Criteria written by hand

The user can paste or type the criteria directly. The right-hand column holds one editable draft that every source of criteria feeds into.

Figure 6: Writing criteria manually. The scoring weights for skills, experience, and education are set on the left and must add up to 100 percent. The amber banner warns the user when a criteria set has not yet been saved.

Step 2b, Criteria generated by AI

When the user selects the AI option, they can add extra context such as emphasis on security or collaboration. Clicking Generate asks the local model to propose a criteria list, which then lands in the same draft area ready for edits.

Figure 7: AI-assisted criteria generation. The additional context box lets HR nudge the model toward specific priorities without writing the full criteria from scratch.

Step 3, Uploading CVs

Candidates are added by dragging PDF or text files into the drop zone. Text is extracted in the browser so no CV leaves the laptop. Duplicate candidates are detected on upload.

Figure 8: The upload step for a chosen job. The header reminds the user of the job, the criteria preview, and the weights that will be used for evaluation.

Step 4, Results

Each candidate receives an overall score and per-dimension scores. The table stays short and scannable; the full reasoning, strengths, and weaknesses appear in a detail view when the user clicks through.

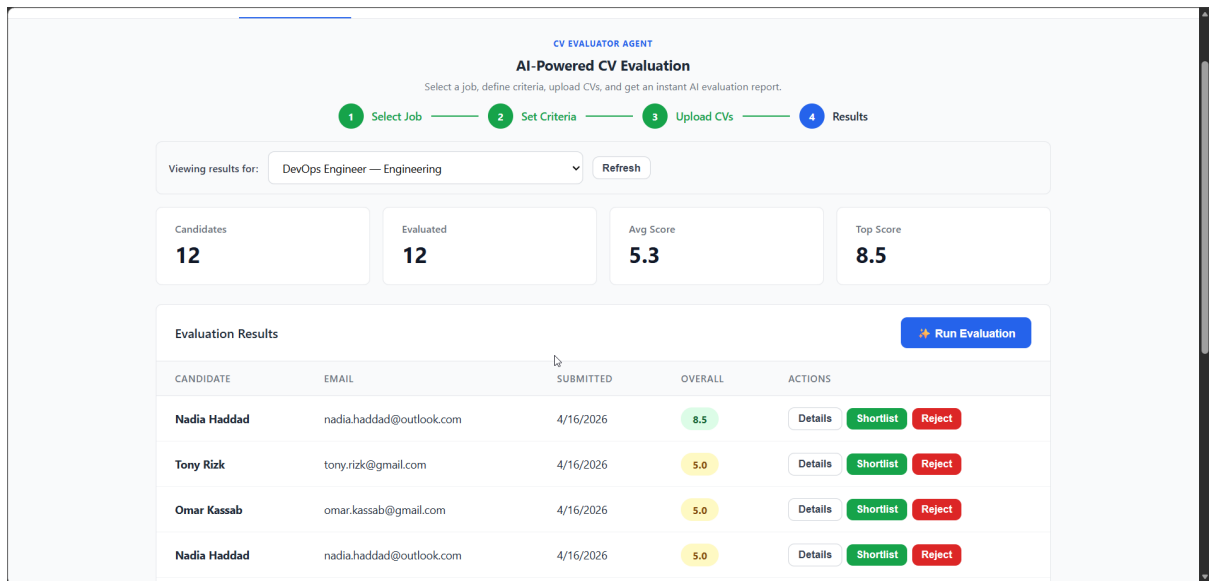


Figure 9: Evaluation results sorted by overall score. From here the user can shortlist a promising candidate, reject one, or open the detail view to read the AI reasoning.

4.4 Shortlist and Interview Tracking

Shortlisted candidates move into a tracking page where HR can mark them as interviewed, hired, or rejected. Status updates happen in place, without reloading the whole page.

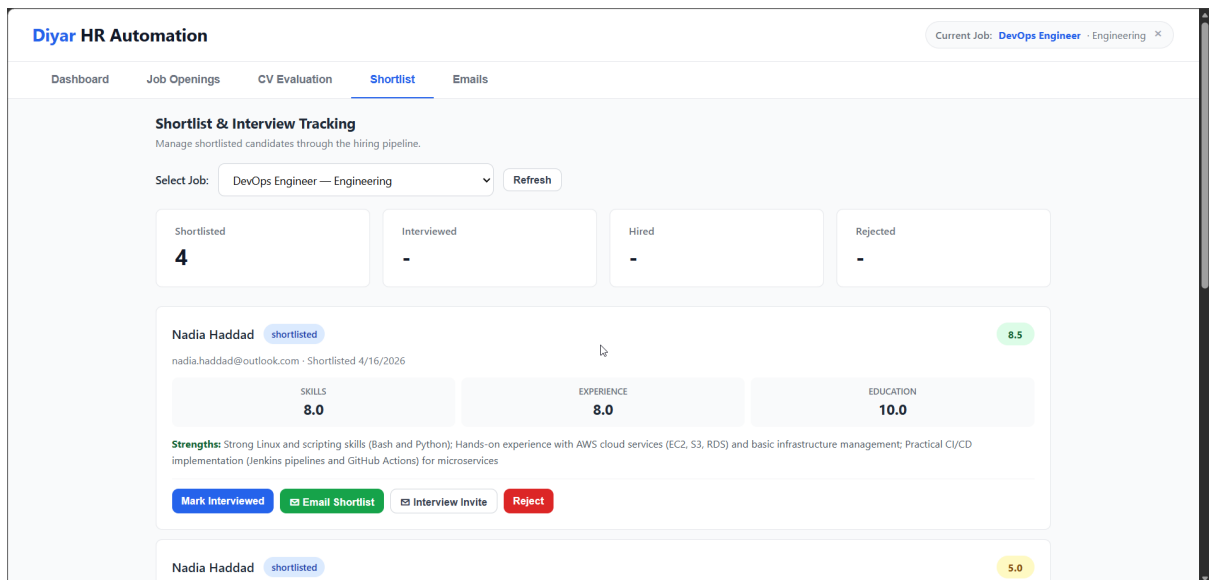


Figure 10: A shortlisted candidate card. The per-dimension scores, strengths summary, and next-step buttons (Mark Interviewed, Email Shortlist, Interview Invite, Reject) are all within one click.

4.5 Emails

Every email the system sends is logged with recipient, subject, timestamp, type, and delivery status. The top of the page shows whether the SMTP connection is healthy, so the HR user can

tell at a glance whether outgoing mail is working.

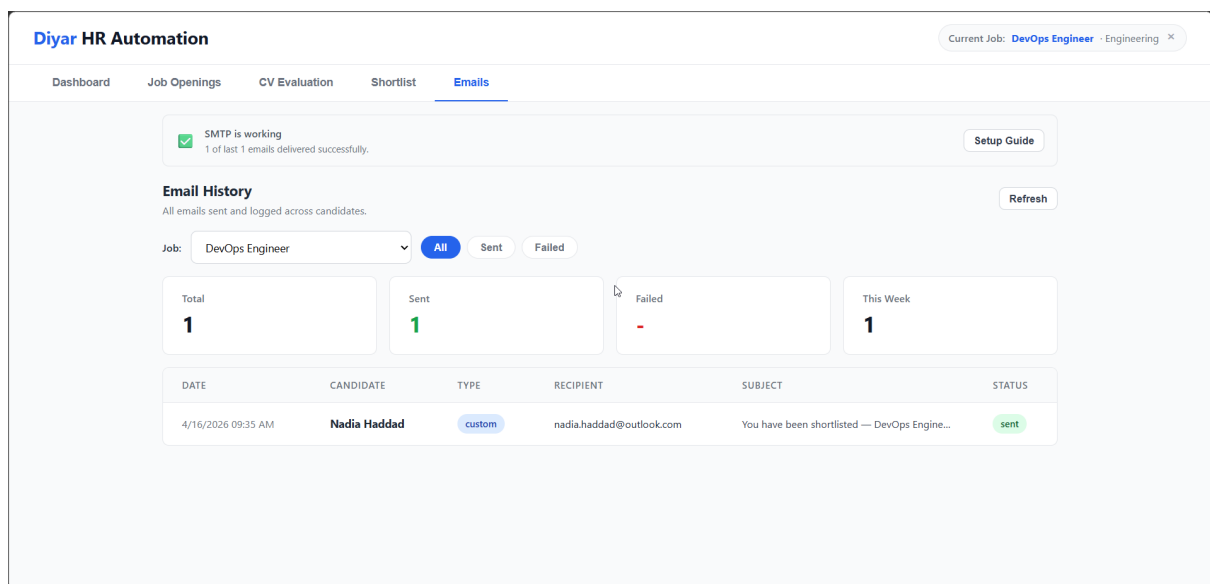


Figure 11: Email history with live SMTP health. Totals, sends, failures, and weekly counts appear as small cards above the log.

5. What Has Been Delivered

The following pieces are functional and in daily use across the demo:

- Job opening lifecycle with AI-assisted description drafting and activation toggles.
- CV intake, in-browser PDF parsing, duplicate detection, and AI evaluation with configurable weights.
- Saved criteria sets per job so previous evaluation setups can be reused.
- Shortlist tracking with four statuses (shortlisted, interviewed, hired, rejected).
- Editable email composer for every candidate-facing message, with logging and SMTP health monitoring.
- Cross-tab selected-job context and a one-command startup script for the full stack.

6. Planned Work Before Finalization

The remaining tasks fall into four groups, listed in the order I plan to tackle them.

6.1 Reliability and Polish

- Tighten the CV parser so multi-column and scanned PDFs extract cleanly.
- Add a test-connection and send-test button to the Emails page so SMTP can be verified in-app.

6.2 Feature Completion

- Finish the LinkedIn publish flow (button already on the page, currently a placeholder).
- Add an email template manager with reusable subject and body templates.
- Extend the dashboard with a time-to-hire metric and a source-of-hire chart.

6.3 Quality and Scoring Improvements

- Split the CV scoring prompt into per-dimension calls for more consistent scores.
- Add a side-by-side comparison view for two or more candidates.

6.4 Handover Readiness

- Short video walkthrough of every tab for new HR users.
- Packaged launcher that installs missing dependencies on a fresh machine in one click.

7. Closing Note

To be clear, this is still a proof of concept and there is meaningful work ahead before it is production-ready. What has been built proves the approach works end to end and gives us a solid base to iterate on, but the items in the previous section are real and will need attention before a full rollout.

This is the progress report for the work done so far, and I am ready to demo the project live on Sunday. Please let me know what time works for you and Mr. Ziad so I can walk you through everything that has been built.